Aaron N. Cutshall, DHA, MSHI

The Lost Science of Set Theory

SQLRV, LLC
HEALTHCARE INFORMATICS
https://www.sqlrv.com

Just who is this guy?

Principal Data Architect, Population Health

Adjunct Instructor

Ambassador Programs

Speaker – various events

B.S. Computer Science

M.S. Computer Information Systems

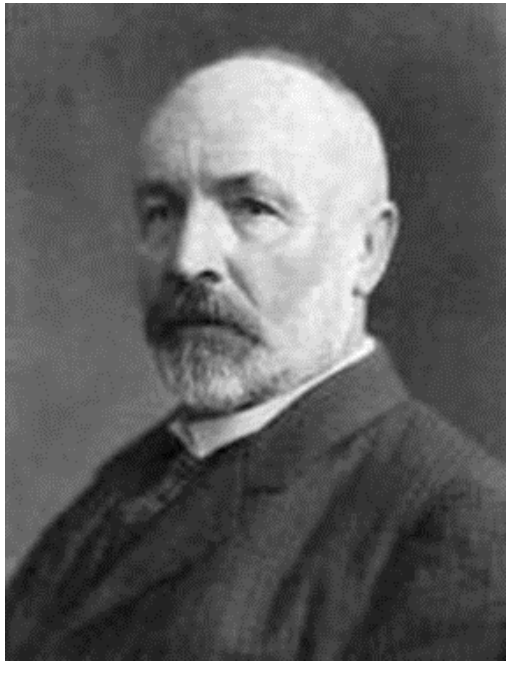
M.S. Health Informatics

Doctor of Healthcare Administration

40 Years

Copyright © 2026, SQLRV LLC - All Rights Reserved

Something to consider…



"A set is a Many that allows itself to be thought of as a One."

– Georg Cantor, 1845-1918

German mathematician and inventor of set theory

Agenda

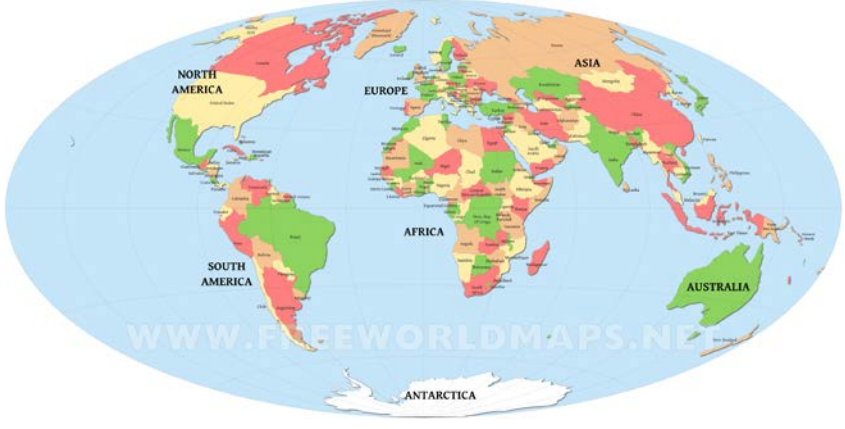
- What is Set Theory?
- Set Operations
- Set Operations in Relational Databases
- Card Tricks
- Database Join Operation
- What does this mean to me?
- Procedural vs Set: What’s the big deal?
- In closing…

What is Set Theory?

- Is a collection of objects
  - Group objects together in a set
  - Treat the set as a single entity
- The objects are known as members of the set
- The members are related to one another in some fashion
- An operation is performed against all members of the set simultaneously
- Sets can be divided into sub-sets
- Multiple sets can be merged into a single set

Some examples in the real world

- Examples of sets:
  - Real Numbers (3.14159, 23, -14.76, 256, -12, etc.)
  - Countries of the world:



  - Fruits (apples, bananas, oranges, peaches, etc.)
  - Database professionals (SQL Server, Oracle, etc.)
  - Deck of cards

Some examples in the real world

- Examples of sub-sets:
  - Integers (-12, 256, 65536, -351, etc.)
  - Countries of Europe:

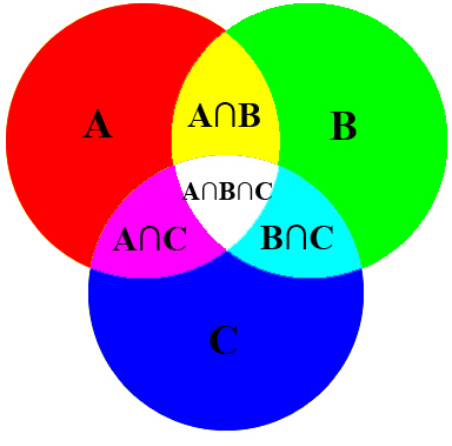
  - Citrus fruits (oranges, lemons, pineapples, etc.)
  - SQL Server professionals
  - Card Suits (spades, clubs, hearts, diamonds)

Why is this important?

- Set theory typically taught only in Computer Science classes (if at all)
- Many DB Developers started out as procedural programmers in Object-Oriented Programming Systems (OOPS)
  - Java
  - Ruby-on-Rails
  - C#
- Therefore, most DB Developers have had very little if any exposure to Relational DB theory

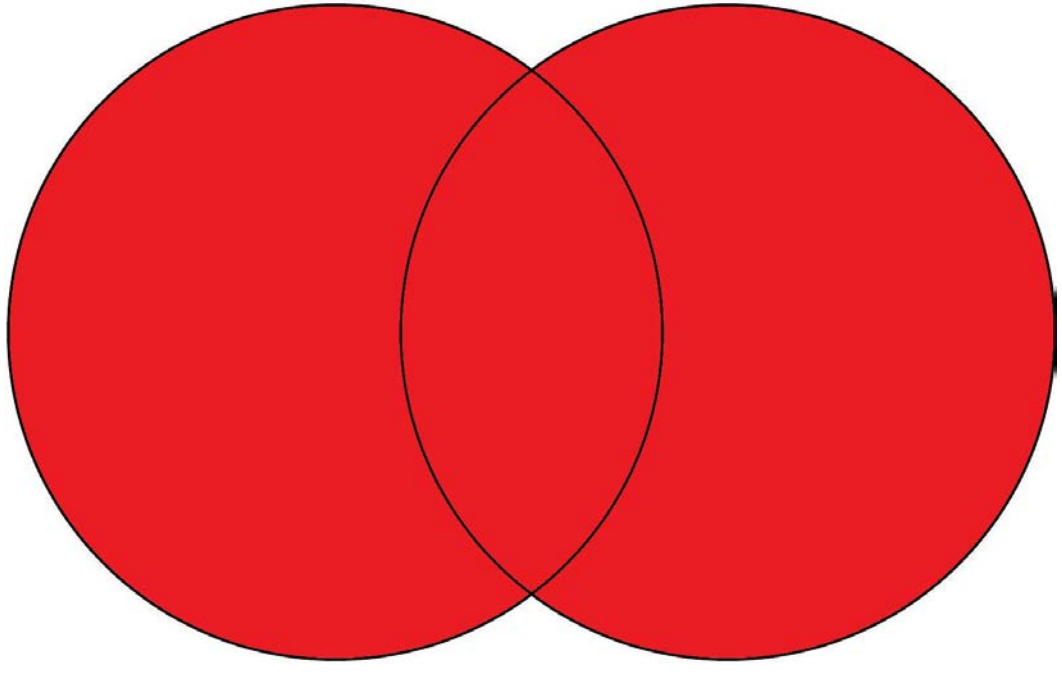
What is the result?

- Set theory is avoided like the plague
- Many DB solutions end up developed in a procedural fashion
- Many are simply just scared of the prospect of handling set-based operations!



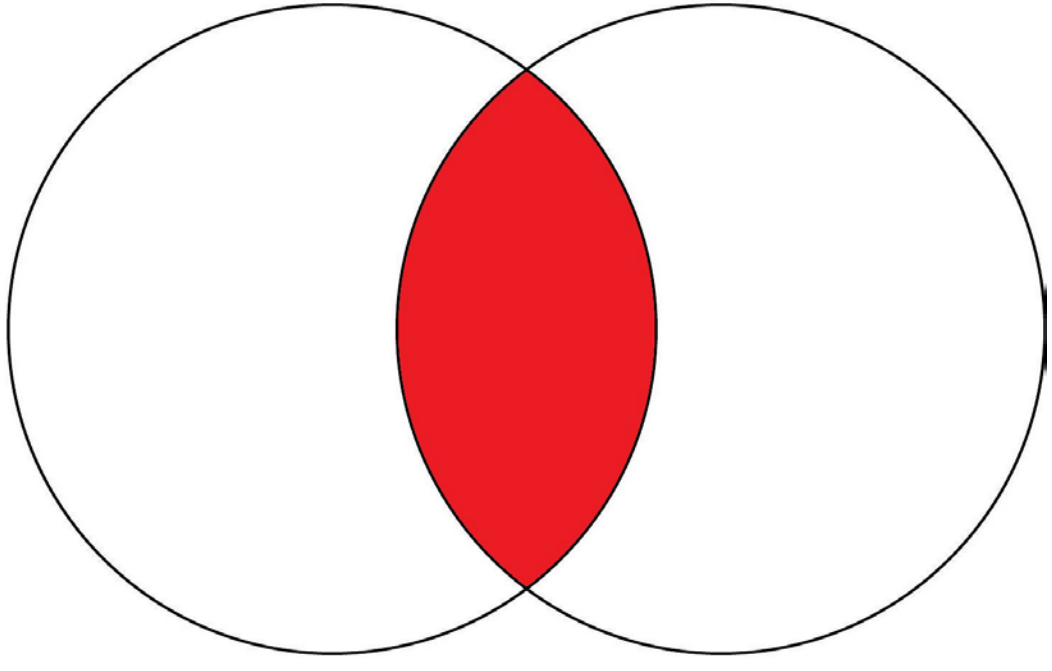
Set Operations

- Union - Combine members of all sets



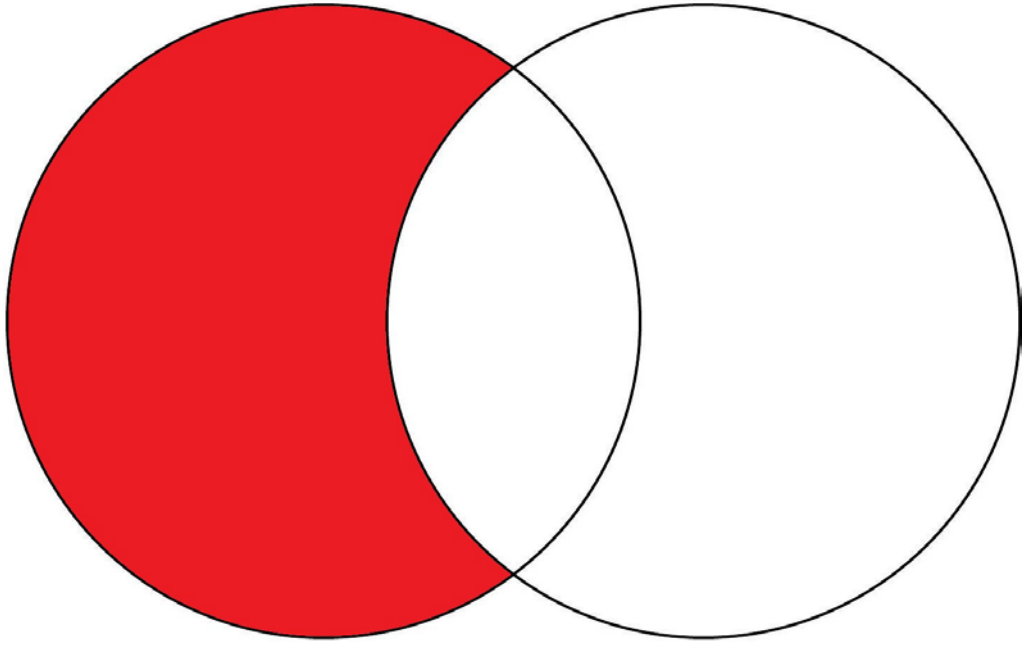
Set Operations

- Intersection – Common members of all sets

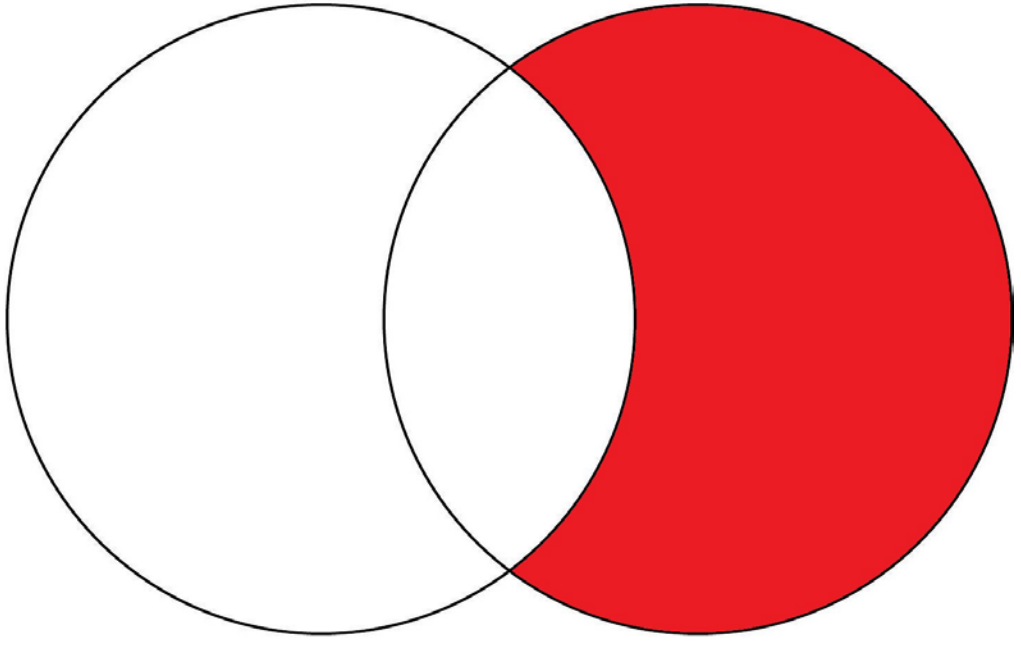


Set Operations

- Difference – Members in one set not in another

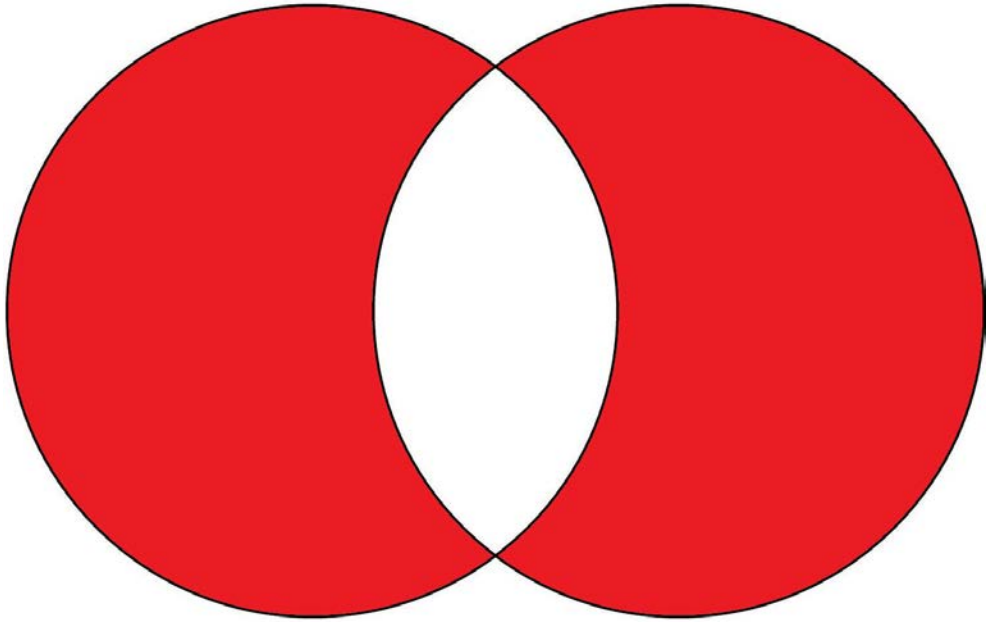


OR



Set Operations

- Symmetric Difference – Members in one set or the other but not in both



Set Operations in Relational Databases

$$R \bowtie S = \{t \cup s \mid t \in R \ \wedge \ s \in S \ \wedge \ Fun(t \cup s)\}$$

$$T = \rho_{x_1/c_1,\ldots,x_m/c_m}(S) = \rho_{x_1/c_1}(\rho_{x_2/c_2}(\ldots \rho_{x_m/c_m}(S)\ldots))$$

$$P = \sigma_{c_1=x_1,\ldots,c_m=x_m}(R \times T) = \sigma_{c_1=x_1}(\sigma_{c_2=x_2}(\ldots \sigma_{c_m=x_m}(R \times T)\ldots))$$

$$U = \pi_{r_1,\ldots,r_n,c_1,\ldots,c_m,s_1,\ldots,s_k}(P)$$

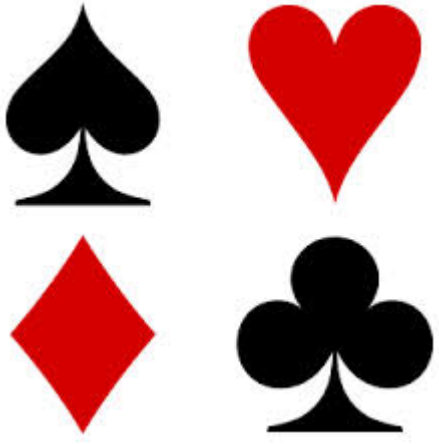
$$(R \bowtie S) \cup ((R - \pi_{r_1,r_2,\ldots,r_n}(R \bowtie S)) \times \{(\omega,\ldots\omega)\})$$

Set Operations in Relational Databases

- Relational Algebra
  - Mathematical relationship between ***members*** of each set
  - Results not in just the members of the sets, but the ***relationships*** between the sets
- Establishing relationship criteria between sets
  - Explicit – Specifically stated relationships
  - Implicit – Implied relationships
- Resultant sets identify all possible relationships between members of the sets based on criteria

Now for some card tricks!

Characteristics of cards



Suits & Color

Face Cards

Numeric Cards

How set theory is used

- Identify sub-sets based upon member characteristics:

```
All cards that are Diamonds.
```

```
All face cards that are Clubs.
```

```
All Aces that are black.
```

```
All numeric cards.
```

How sets are used in a Relational DB

- Query data based upon criteria:

```
SELECT Card
FROM Room
WHERE Suit = ‘Diamond’;
```

```
SELECT Card
FROM Room
WHERE Suit = ‘Club’
AND ISNUMERIC(CardNbr) = 0; -- Face Card
```

```
SELECT Card
FROM Room
WHERE Color = ‘Black’
AND CardNbr = ‘A’;
```

```
SELECT Card
FROM Room
WHERE ISNUMERIC(CardNbr) = 1; -- Numeric Card
```

INNER JOIN (Intersect)

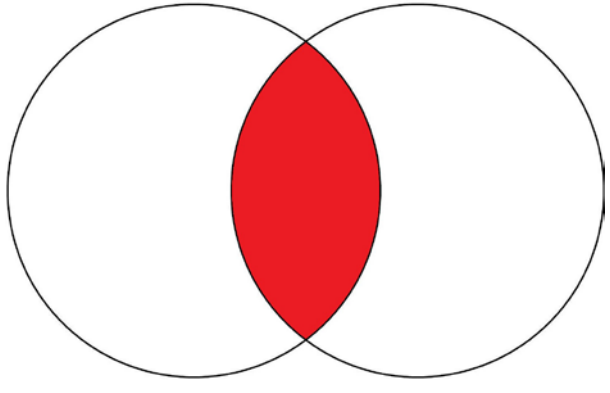
- Most frequently used join
- Produces a set of records which match both tables

Pure Set:

```
All cards from both Left Side
and Right Side that have only
the Hearts suit.
```

Relational Set:

```
SELECT L.Card, R.Card
FROM LeftSide L
    INNER JOIN RightSide R
        ON L.CardNbr = R.CardNbr
WHERE L.Suit = ‘Hearts’;
```



LEFT JOIN (Difference & Intersect)

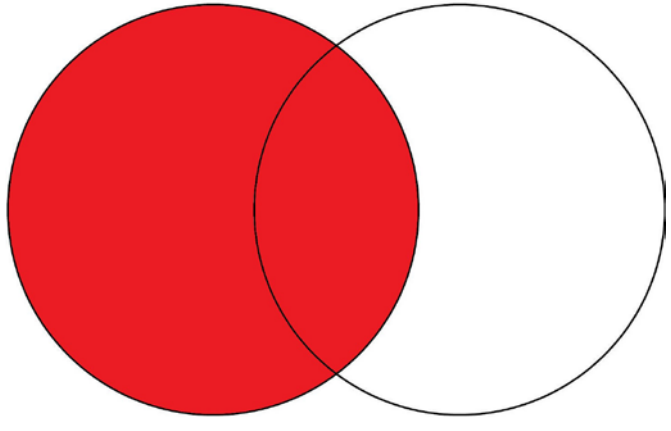
- Second most frequently used join
- All records from left table and matching ones from right table

Pure Set:

```
All cards from both Left Side
and Right Side that have red
suits.
```

Relational Set:

```
SELECT L.Card, R.Card
FROM LeftSide L
    LEFT JOIN RightSide R
        ON L.CardNbr = R.CardNbr
WHERE L.Color = ‘Red’;
```

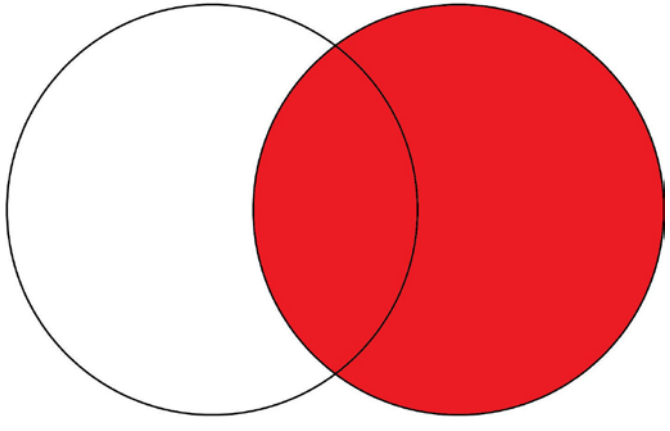


RIGHT JOIN (Intersect & Difference)

- Rarely Used – Similar to LEFT JOIN in reverse
- All records from right table and matching ones from left table

Pure Set:

```
All cards from both Right Side
and Left Side that are face
cards.
```



Relational Set:

```
SELECT L.Card, R.Card
FROM LeftSide L
    RIGHT JOIN RightSide R
        ON R.CardNbr = L.CardNbr
        AND ISNUMERIC(R.CardNbr) = 0;
```

FULL OUTER JOIN (Union)

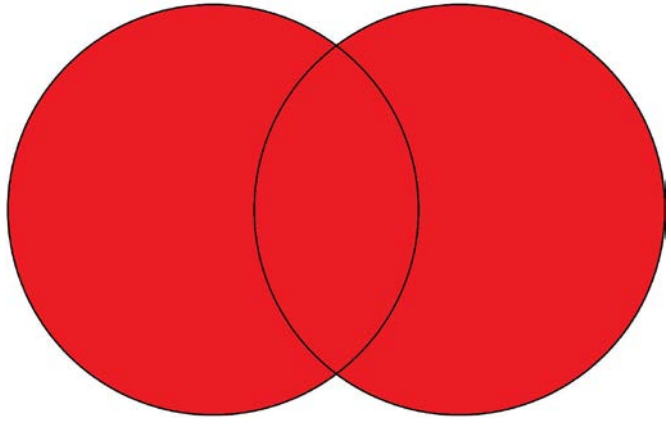
- Lesser value – Union of LEFT JOIN and RIGHT JOIN
- All records from left and right tables regardless of matches

Pure Set:

```
All cards from both Left Side
and Right Side.
```

Relational Set:

```
SELECT L.Card, R.Card
FROM LeftSide L
    FULL OUTER JOIN RightSide R
        ON L.CardNbr = R.CardNbr;
```



Database Join Operation

- Logical vs Physical
  - Query join is a logical operation
  - Physical join may be implemented differently on different systems
- Join Operation Selection Criteria
  - System constraints (memory, disk space, etc.)
  - Table structure
  - Indexing structure
  - Data volume
  - The set of values in the participating columns
- Actual join operation performed in memory
  - Increased performance
  - More memory, better performance

Database Join Operation

- Adaptable to changing conditions
  - Changes in query or join criteria
  - Changes in data and/or data relationships (via updated statistics)
  - Changes in hardware status (memory, storage, etc.)
  - Previous executions (saved execution plans)
- Join Operation
  - Join Operation selected is dependent upon type of query, tables sizes, indexes available, etc.
  - Engine attempts to select proper join operation for given circumstances

Database Join Operation

- Merge
  - Both tables are joined on sorted indexed columns
  - SQL Server scans once through the sorted inputs and merges the data together
- Nested Loop
  - When one of the tables (inner table) contains few records compared to the number of records in the outer table
  - Nested loops scan once through each inner table row and searches for corresponding rows in the outer table

Database Join Operation

- Hash
  - When large, unsorted, non-indexed columns are used in the join condition
  - Engine creates a hash table by applying a hash function to the join attribute of each row in the smaller table
  - Sort of like creating a temporary index on the fly
  - Scans the larger relation and finds the relevant rows by looking at the hash table
- Note
  - One join operation no more costly than another except when used improperly

Procedural vs Set: What's the big deal?

- Procedural (or Object-Oriented) processing
  - Data is very abstract
    - Structure or organization of the data is unimportant
    - Separated by different layers of abstraction
  - Concerned with what to do and how to do it
    - Step-by-step
    - Statements, Loops, Object manipulation, etc.
    - Different logic paths depending upon data attributes
    - Record by record processing (RBAR)
  - Provides total control
    - You know exactly what it will do
    - Execution method will be consistent each and every time

Procedural vs Set: What's the big deal?

- Set-based processing
  - Describes what should be done
    - Indicates the operation
    - Identifies the conditions
  - Allows the engine to determine an optimal method to obtain results based upon:
    - Data structure, organization and volume
    - Statistics from previous runs
  - Frees developer from mechanical/specific details
  - Allows for changing conditions without altering query

Procedural vs Set: What's the big deal?

- Set-based processing
  - RDBMS is optimized to select or manipulate records sets as a whole (often at the expense of procedural operations)
  - Requires a paradigm shift in thinking from HOW to do something to WHAT you want to do.
  - Challenges you to consider what may be happening under the hood.
  - "Stop thinking about what you want to do to a row. Instead, think about what you want to do to a column." – Jeff Moden

Example Scenario

- Process Healthcare Measures for multiple hospitals
  - Measures are typically ratios (numerator over denominator)
  - Ex. Measure: identify diabetic patients (denominator) who have an LDL-C < 100 mg/dL (numerator)
  - Calculation session may be for as many as 70+ hospitals (120k+ patients each)
  - Each hospital may require 40 or more measures
- Restrictions
  - Each measure must be calculated independently
  - Each measure may be processed for one or more hospitals
  - Calculated over multiple concurrent time-periods (typically weekly, monthly, quarterly, or annually)
  - Same data may be used for multiple measures

Procedural Example

- Start Job (Call SP to create job record)
- Load temp table for patients in each hospital from a very complex view construct (multiple nested views with many referencing the same tables)
- For each time period (weekly, monthly, quarterly, annually)
  - For each row in the temp table (via cursor)
    - Calculate Reporting Period Begin/End Date for hospital
    - Start Program Session (Call SP to create record)
    - For each measure a hospital requires (via cursor)
      - Clear working tables
      - Execute measure stored procedure per Hospital
      - Copy working tables to permanent tables
    - End Program Session (Call SP to update record)
- End Job (Call SP to update session job)

Procedural Example

- Example of a Job_Start:
  - Determine @JobId
  - Call SP Job_Insert
    - INSERT INTO tbl_Job VALUES (@JobId, @Parameter1, @Parameter2, …);

- Example of a Job_End:
  - Call SP Job_Update
    - UPDATE tbl_Job SET StopTime = GETDATE() WHERE JobId = @JobId;

Set-Based Example

- SELECT Facilities & Measures INTO temporary table FROM simplified query for all Hospitals with all patients and all time periods
- Initialize measure counter to 1
- Loop through the number of measures
  - Execute measure stored procedure
  - Copy results to permanent tables
  - Increment measure counter

Why is this better?

- Eliminated nested SP calls which resulted in a single record insert/update
- Loaded all data into temp table in one query
- Grouped records allow for all hospitals per measure instead of just one at a time
- Eliminated performance killing cursors
- A loop was still needed because SPs cannot be executed in a set
- Loop iterations now from 11,200 (4*70*40) to just 40!

In closing…

- Procedural programming is not set-based
  - Intended for greater control over processing
  - More efficient with loops, recursion, and string-based manipulation
  - Better with small amounts of data at a time
- Relational database engines are set-based
  - More efficient when processing large data sets
  - Can generate dynamic and efficient execution plans
  - Outperforms procedural processing with large data sets
  - Unfortunately, not everything can be done as a set
- ***<u>Use the right tool for the right job!</u>***

In closing…

- Change your thinking to set-based operations
- Consider set-based operations in your logic
- Review cursors and loops and look for set-based methods
- Give up mechanical control for greater overall control
- Work WITH the engine – not against it
- Yoda: “Unlearn what you have learned you must!"

In closing…

- Barely scratched the surface
- Simple examples shown here (only two simple sets)
- Much more complex operations are typical
  - Multiple tables in joins of various types
  - Complex relationship criteria
- A whole course could be given on this subject alone
- An additional set of courses could be given on transforming procedural processes into set-based processes
- In progress: *A Practical Guide to Set-Based Queries* from Apress Books

Useful resources

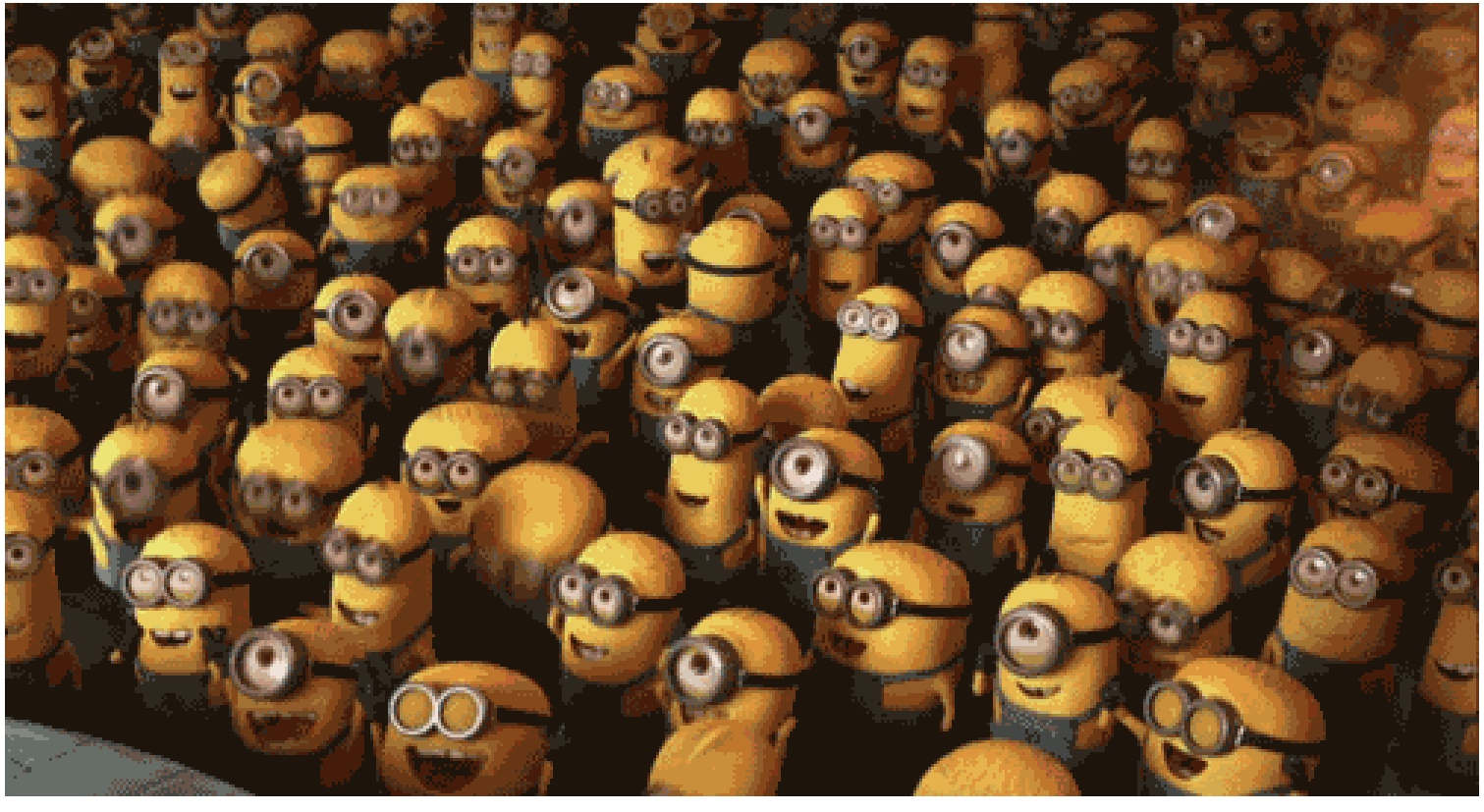
- Understanding set-based logic
  - http://www.codinghorror.com/blog/2007/10/a-visual-explanation-of-sql-joins.html
  - http://sbuweb.tcu.edu/bjones/20263/Access/AC101_FundamentalsDB_Design.pdf
  - http://sqlmag.com/t-sql/t-sql-foundations-thinking-sets
  - http://weblogs.sqlteam.com/jeffs/archive/2007/04/30/thinking-set-based-or-not.aspx

Useful resources

- Converting procedural code to set-based
  - http://sqlmag.com/t-sql/programming-sql-set-based-way
  - http://www.exacthelp.com/2013/02/write-sql-queries-in-set-based-approach.html
  - https://www.simple-talk.com/sql/performance/writing-efficient-sql-set-based-speed-phreakery/
- Other:
  - https://stevestedman.com/2015/05/tsql-join-types-poster-version-4-1/

Thanks to our Sponsors!

Without them, we wouldn’t be here!!



Copyright © 2026, SQLRV LLC - All Rights Reserved

Questions & Comments

BONUS:

A **TON** of free eBooks from RedGate and others!

PRESENTATION FEEDBACK:

- Your thoughts needed
- Improve presentations
- Make this event even more valuable!!!

Aaron N. Cutshall

www.linkedin.com/in/sqlrv

aaron@sqlrv.com

www.sqlrv.com

www.youtube.com/@sqlrv

Copyright © 2026, SQLRV LLC - All Rights Reserved